
Automotive QA Intelligence

System Architecture & Technical Documentation

Comprehensive Technical Reference for the Offline RAG-Powered
Automotive Quality Analytics & Diagnostics Platform

Document Version: **1.1**

Date: **June 12, 2026**

Classification: **CONFIDENTIAL — INTERNAL**

Prepared by: **Senior Solutions Architect**

Table of Contents

1. System Overview
2. End-to-End Architecture
3. Data Flow & Processing Pipeline
4. Component Interactions
5. Backend Workflow
6. Database Architecture
7. API & Query Flow
8. AI / ML Pipeline
9. Retrieval Pipeline (SQL + BM25 + FAISS + RAG)
10. Frontend Architecture
11. Report Generation Engine
12. Deployment Architecture
13. Scalability Considerations
14. Performance Optimizations
15. Security Considerations
16. Error Handling & Resilience
17. Future Improvements
18. Technology Stack Summary

1. System Overview

Automotive QA Intelligence is a fully offline, enterprise-grade quality analytics and diagnostics platform designed for automotive field technical information report (FTIR) analysis. The system enables technicians and quality engineers to perform natural language queries against a structured database of historical vehicle failure records, leveraging a hybrid Retrieval-Augmented Generation (RAG) architecture that combines SQL-based structured search, FAISS vector similarity search, and local Large Language Model (LLM) inference.

1.1 Core Objectives

- **Offline-First Architecture:** Operates entirely without internet connectivity using local LLM inference (Phi-3 Mini 4K via llama.cpp) and local FAISS vector indexing.
- **Hybrid Retrieval:** Combines SQL keyword search, FAISS cosine similarity vector search, and NLP entity extraction for precise multi-strategy information retrieval.
- **Intelligent Query Routing:** An NLP pipeline classifies user intent (SEARCH, ANALYTICS, VISUALIZE, COMPARE, REPORT) and dispatches queries to the optimal processing backend.
- **Dynamic Analytics:** LLM-generated SQL queries enable ad-hoc analytics without pre-defined report templates.
- **Automated Report Generation:** Monthly PDF and DOCX quality reports are compiled dynamically from database aggregations.
- **Enterprise UI:** A Streamlit-based web interface with multi-session chat, real-time data visualization (Plotly), and citation-backed responses.

Key Differentiator: Unlike cloud-based AI platforms, this system operates entirely offline with no API keys, no external network calls, and no data egress — making it suitable for air-gapped automotive manufacturing environments where data confidentiality is paramount.

1.2 System Specifications

Parameter	Specification
LLM Engine	Microsoft Phi-3 Mini 4K Instruct (Q4 GGUF, 3.8B params)
Embedding Model	all-MiniLM-L6-v2 (SentenceTransformers, 384-dim)
Vector Index	FAISS IndexFlatIP with IDMap (Cosine Similarity)
Database	SQLite 3 with WAL mode + Materialized Views
Frontend	Streamlit 1.x with custom enterprise CSS theme
Visualization	Plotly Express + Plotly Graph Objects
LLM Runtime	llama-cpp-python with CUDA GPU offloading
Report Formats	PDF (ReportLab) + DOCX (python-docx)
Deployment	Single-node local / on-premise (Windows/Linux)

2. End-to-End Architecture

The system follows a layered architecture pattern with clear separation of concerns across six layers: Presentation, Application Logic, Intelligence (AI/ML), Data Access, Storage, and Infrastructure.

Figure 2.1 — Layered System Architecture

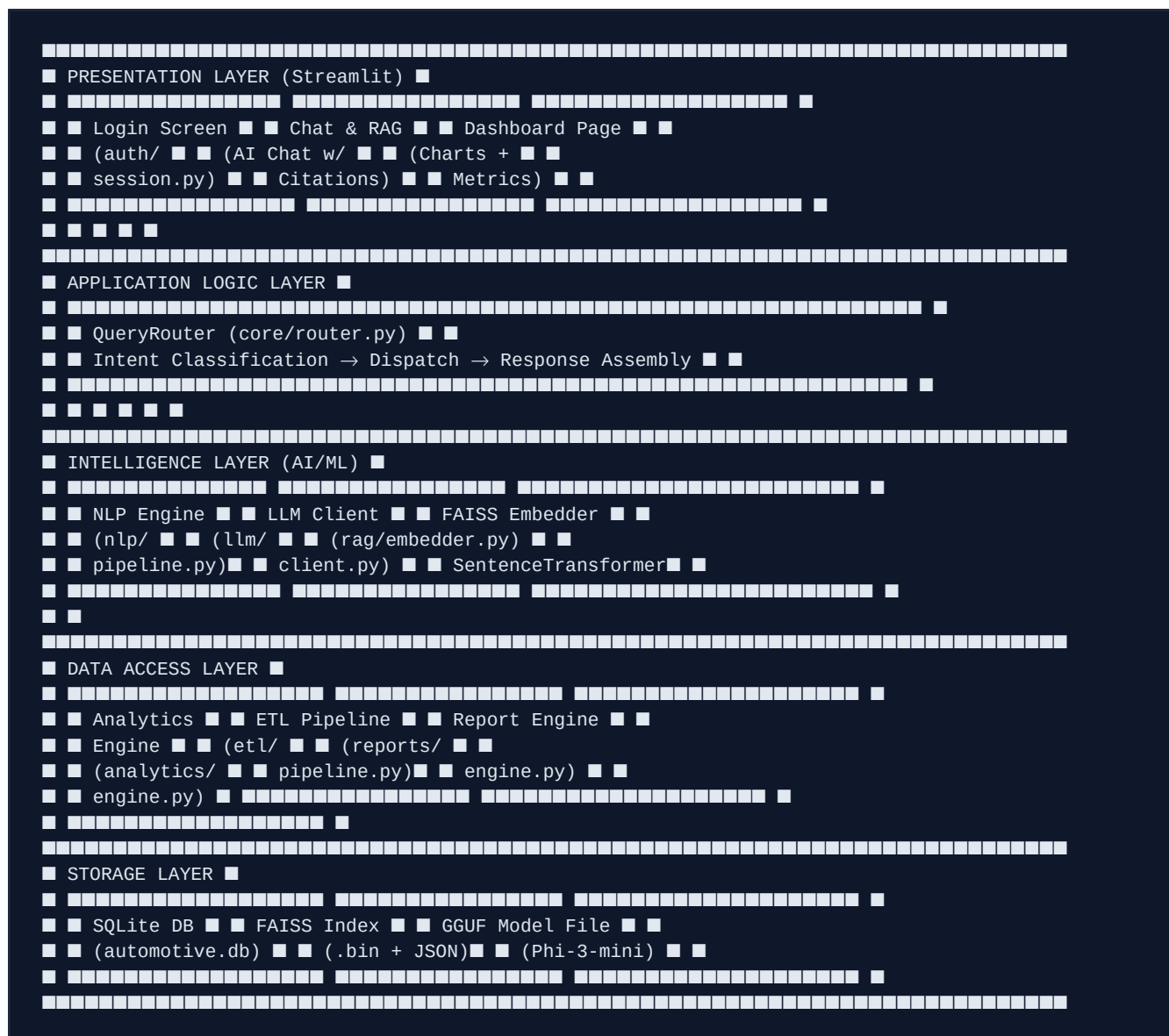


Figure 2.1: Six-layer architecture showing clear separation between Presentation, Application Logic, Intelligence, Data Access, and Storage layers.

2.1 Module Directory Structure

Figure 2.2 — Project Directory Structure

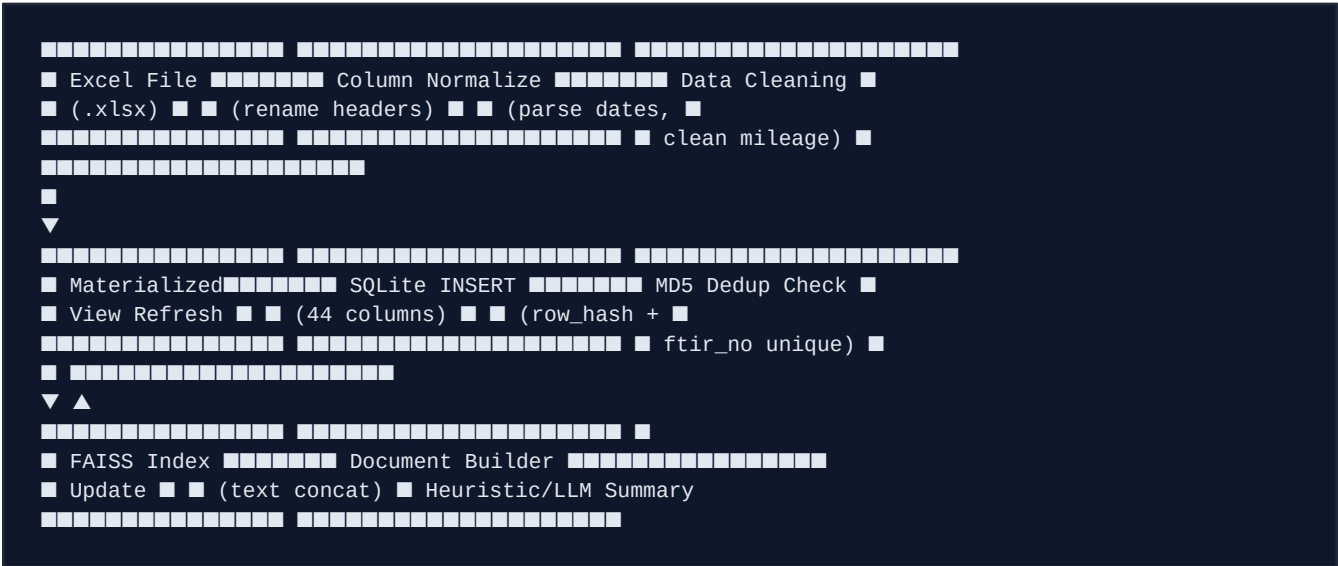
```
automotive_qa/
  app/ # Streamlit frontend (main.py)
  auth/ # User session management (session.py)
  core/ # Central infrastructure
    router.py # Query routing & dispatch engine
    singletons.py # Cached resource loaders (DB, LLM, FAISS)
    cache.py # L1 (RAM) + L2 (SQLite) query cache
    paths.py # Absolute path resolver
  etl/ # Data ingestion & transformation
    pipeline.py # Excel → SQLite ETL with deduplication
  nlp/ # Natural language processing
    pipeline.py # Intent classifier + entity extractor
  rag/ # Retrieval-Augmented Generation
    embedder.py # FAISS index manager + vector search
    document_builder.py # Structured text builder for embeddings
  llm/ # Local LLM inference
    client.py # Phi-3 GGUF streaming client (llama.cpp)
  analytics/ # SQL analytics engine
    engine.py # 12+ parameterized analytics queries
    graph_selector.py # Dynamic chart type inference
    views.py # Materialized view refresh logic
  viz/ # Plotly visualization library
    charts.py # 6 chart types with premium theming
  reports/ # Document generation
    engine.py # PDF + DOCX report builder
  data/ # Persistent data store
    automotive.db # SQLite database
    faiss_index.bin # FAISS vector index
    vector_metadata.json # Index metadata
    inbox/ # Manual ingestion drop folder
  models/ # GGUF model weights
    Phi-3-mini-*.gguf
  schema.sql # Database DDL (tables + indexes)
  setup.py # Automated project bootstrapper
  requirements.txt # Python dependencies
```

3. Data Flow & Processing Pipeline

3.1 Ingestion Data Flow (ETL)

Data enters the system through Excel (.xlsx) files containing FTIR records from field operations. The ETL pipeline performs column normalization, data cleaning, deduplication via MD5 hashing, computed field generation, heuristic/LLM-based summarization, and materialized view refresh — all within a single atomic transaction.

Figure 3.1 — ETL Ingestion Pipeline

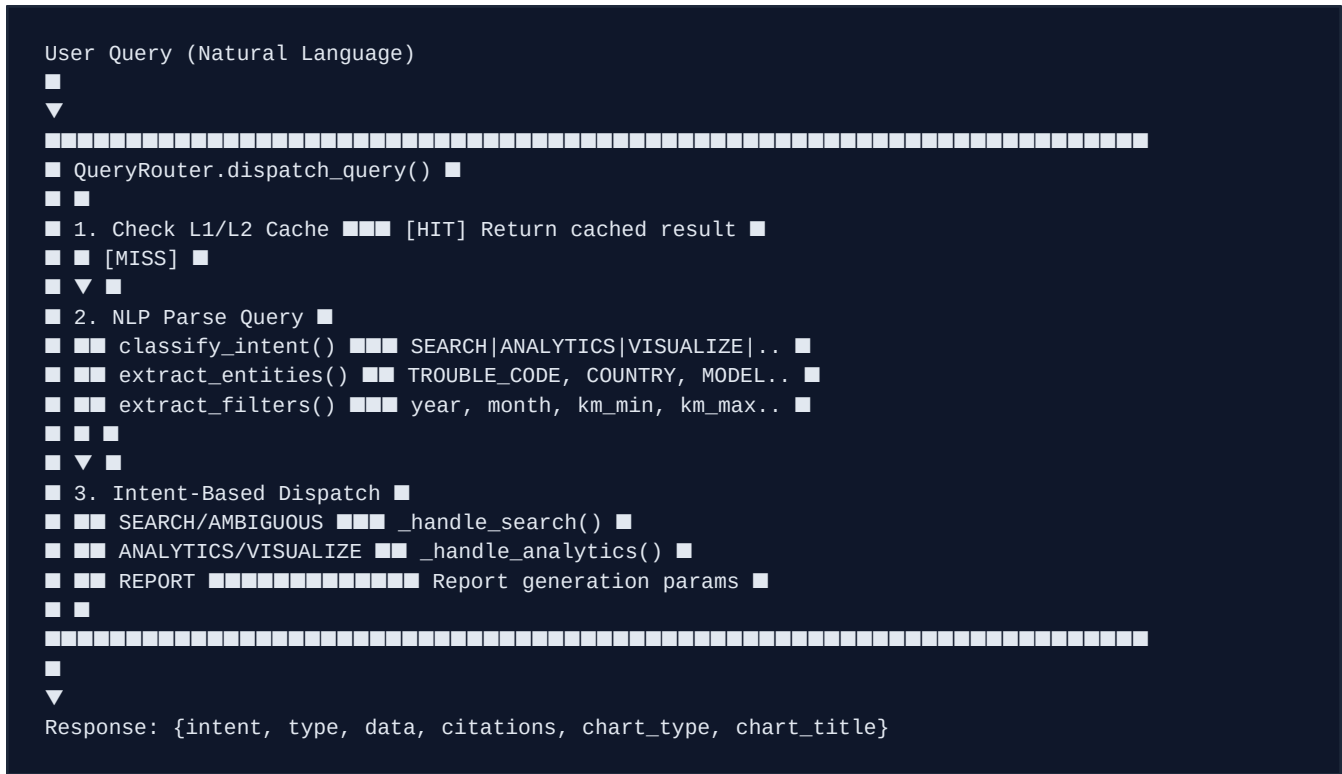


3.2 ETL Processing Steps

Step	Module	Operation	Details
1	etl/pipeline.py	Read Excel	pd.read_excel() with column rename normalization
2	etl/pipeline.py	Clean & Parse	Date parsing (4 formats), mileage integer extraction
3	etl/pipeline.py	Deduplication	MD5(ftir_no subject complaint) + UNIQUE constraint
4	etl/pipeline.py	Computed Fields	using_km_int, report_year/month, is_resolved, has_sbpr
5	etl/pipeline.py	Summarization	LLM 2-sentence summary or heuristic fallback
6	analytics/views.py	MV Refresh	5 materialized views rebuilt in single transaction
7	rag/document_builder.py	Text Build	Concatenate 6 key fields into embedding document
8	rag/embedder.py	Vectorize	SentenceTransformer encode + FAISS IndexIDMap upsert

3.3 Query Processing Data Flow

Figure 3.2 — Query Processing Pipeline



4. Component Interactions

The following diagram illustrates the runtime interaction between all major system components during a typical user query lifecycle. Components communicate via function calls within a single Python process.

Figure 4.1 — Component Interaction Diagram



4.1 Singleton Resource Management

All expensive resources are loaded once and cached using Streamlit's `@st.cache_resource` decorator in **core/singletons.py**. This ensures the FAISS index (~9.5 MB), SentenceTransformer model, and Phi-3 GGUF model (~2.4 GB) are loaded into GPU VRAM/RAM only once across all Streamlit reruns.

Singleton	Module	Cached Resource	Lifecycle
<code>get_db_connection()</code>	<code>core/singletons.py</code>	SQLite DB path (WAL mode verified)	App lifetime
<code>get_embedder()</code>	<code>core/singletons.py</code>	SentenceTransformer + FAISS index + metadata	App lifetime
<code>get_llm()</code>	<code>core/singletons.py</code>	Phi-3 GGUF loaded via llama-cpp (GPU offload)	App lifetime

Singleton	Module	Cached Resource	Lifecycle
get_ingestion_tracker()	core/singletons.py	Thread-safe ingestion status tracker	App lifetime

Step	Operation	Module
Step 8	Response cached to L1 RAM + L2 SQLite	core/cache.py
Step 9	Chat message persisted to chat_history table	auth/session.py

6. Database Architecture

The system uses SQLite 3 with Write-Ahead Logging (WAL) mode for concurrent read/write access. The schema consists of 10 tables: 1 core data table, 4 application tables, and 5 materialized view tables.

6.1 Core Schema — records Table

The **records** table stores all FTIR data with 44 columns including 6 computed fields. It serves as the single source of truth for all analytics, search, and report generation.

Column Group	Columns	Purpose
Identity	id, sbpr_no, ftir_no, row_hash	Primary key, deduplication
Temporal	ftir_report_date, reply_date, date_registered, date_of_incident	Date tracking
Vehicle	product_model_code, sales_model_code, vin, engine_no, transmission_no	Vehicle identification
Classification	segmentation, status, quality, fc_ok	Categorical grouping
Geography	outbreak_country, reported_company, issued_company, manufacturer_factory	Location & dealer tracking
Technical	subject, customer_complaint, trouble_code_complaint, trouble_code_defect	Failure description
Diagnostics	checked_contents, checked_results, repair_status, repair_contents	Inspection & repair data
Resolution	problem_solved, action_judgement, causal_parts_no, causal_parts_name	Root cause & fix
Computed	using_km_int, report_year, report_month, is_resolved, has_sbpr, summary	Derived analytics fields

6.2 Application Tables

Table	Purpose	Key Columns
users	Passwordless user authentication	id, username, created_at
chat_sessions	Multi-thread conversation management	id, user_id, title, created_at
chat_history	Persistent message storage	id, user_id, session_id, role, content
query_cache	L2 query result cache with TTL	query_hash, user_id, result_json, expires_at
embedding_cache	Persistent embedding vector cache	text_hash, embedding_blob (BLOB)

6.3 Materialized Views

Five materialized view tables are pre-computed from the records table and refreshed atomically after every ETL ingestion. These provide O(1) access to common dashboard aggregations without runtime computation.

View Table	Aggregation	Refresh Trigger
mv_country_month	Record count by country, year, and month	Post-ETL ingestion
mv_trouble_codes	Frequency count by trouble code	Post-ETL ingestion
mv_dealer_summary	Record count by reporting dealer/company	Post-ETL ingestion
mv_quality_dist	Distribution count by quality rating	Post-ETL ingestion

6.4 Index Strategy

Nine B-tree indexes cover 95%+ of filter patterns used in analytics and search queries:

Index Name	Column	Primary Use Case
idx_outbreak_country	outbreak_country	Country-based filtering & grouping
idx_product_model_code	product_model_code	Model-specific analytics
idx_segmentation	segmentation	Engine/Transmission category filters
idx_trouble_code_complaint	trouble_code_complaint	DTC frequency analysis
idx_status	status	Status-based filtering
idx_quality	quality	Quality distribution queries
idx_repair_status	repair_status	Repair status analytics
idx_using_km_int	using_km_int	Mileage range queries
idx_report_year	report_year	Temporal analytics

7. API & Query Flow

The system does not expose a REST API — all interactions occur through the Streamlit UI. However, the QueryRouter acts as an internal API gateway that routes classified intents to the appropriate processing backend.

7.1 Internal Query API Contract

The QueryRouter returns a standardized response dictionary:

Field	Type	Description
intent	str	Classified intent: SEARCH, ANALYTICS, VISUALIZE, COMPARE, REPORT, AMBIGUOUS
type	str	Response type: text_stream, table_stream, table_only, report
data	mixed	Messages (list of dicts) for LLM, or DataFrame, or report params
citations	list[dict]	Referenced FTIR records with ftir_no, subject, company, country
chart_type	str	Plotly chart type: horizontal_bar, line, donut, histogram, radar, grouped_bar
chart_title	str	Generated chart title string
sql_query	str	The actual SQL query executed (for transparency)

7.2 Intent Classification Rules

Intent	Score	Trigger Keywords	Handler
VISUALIZE+EXPLAIN	4	(chart graph plot) AND (why explain cause)	_handle_analytics()
VISUALIZE	3	chart, graph, plot, visualize	_handle_analytics()
COMPARE	3	compare, vs, versus, difference between	_handle_analytics()
REPORT	3-6	monthly report, generate report, export + year	Report params
ANALYTICS	2	top, count, how many, total, frequency, average	_handle_analytics()
SEARCH	3	similar, find, cases like, search, lookup	_handle_search()
AMBIGUOUS	0	No keyword match (fallback to search)	_handle_search()

8. AI / ML Pipeline

8.1 NLP Processing Pipeline

The NLP pipeline in `nlp/pipeline.py` performs three sequential tasks: intent classification, named entity recognition (NER), and filter extraction. It uses a dual-mode approach: spaCy with custom EntityRuler patterns as primary, with regex-based fallback.

Entity Type	Recognition Method	Pattern Example	Database Validation
TROUBLE_CODE	Regex: <code>[PBCU]\d{4}</code>	P0500, C0035	No (format-based)
PRODUCT_MODEL	Regex: <code>Y[A-Z0-9]{5,9}</code>	YNC412, YFBP51	Yes (cross-checked with DB models)
COUNTRY	Lookup list from DB	India, Brunei, Nepal	Yes (loaded at init from records)
SALES_MODEL	Regex: <code>[A-Z]{3}\d{3,4}</code>	ERT701, ATM412	No (format-based)
VIN	Regex: <code>MA3[A-Z0-9]{14}</code>	MA3FJEB1S00123456	No (format-based)
FTIR_NO	Regex: <code>FTIR\d{4}\d{4}</code>	FTIR/2024/1018	No (format-based)

8.2 Local LLM Inference Engine

The LLM subsystem uses **Microsoft Phi-3 Mini 4K Instruct** (3.8B parameters, Q4 quantized GGUF format) running locally via `llama-cpp-python` with full CUDA GPU layer offloading. The model serves three distinct roles:

Role	Function	Temperature	Max Tokens	Stop Tokens
Chat Response	<code>generate_chat_stream()</code>	0.1	300	<code>< end ></code> , <code>< user ></code> , <code>< system ></code>
Record Summary	<code>generate_summary()</code>	0.1	150	<code>\n, Summary:</code> , <code>Issue:</code>
SQL Generation	<code>query_via_llm()</code>	0.1	500	<code>< end ></code> , <code>< user ></code>

The Phi-3 native chat template format is used for all prompts: `<|system|>...<|end|> <|user|>...<|end|> <|assistant|>`

8.3 LLM-Generated SQL with Self-Healing

The AnalyticsEngine in `analytics/engine.py` implements a sophisticated LLM-to-SQL pipeline with automatic error recovery. When the LLM generates an invalid SQL query, the error message is appended to the prompt context, and the LLM is asked to self-correct — up to 4 retry attempts.

Figure 8.1 — LLM SQL Self-Healing Pipeline

```
Attempt 1: LLM generates SQL → Execute
■
■■■ [SUCCESS] → Return DataFrame
■
■■■ [SQL ERROR] → Append error to prompt context
■ ■
■ ▼
■ Attempt 2: LLM re-generates → Execute
■ ■
■ ■■■ [SUCCESS] → Return DataFrame
■ ■
■ ■■■ [SQL ERROR] → Append error...
■ ■ ■
■ ■ ▼
■ ■ Attempt 3 → Attempt 4
■ ■
■ ■■■ [MAX RETRIES] → Raise exception
■ ■
■ ▼
■ Static fallback router
■ (12 parameterized queries)
```


9. Retrieval Pipeline (SQL + FAISS + RAG)

The SEARCH intent triggers a three-stage hybrid retrieval pipeline that combines structured SQL filtering, FAISS vector similarity search, and LLM-powered response generation with source citations.

9.1 Three-Stage Hybrid Retrieval

Figure 9.1 — Three-Stage Hybrid Retrieval Pipeline

```
STAGE 1: SQL Pre-Filter (Metadata Whitelisting)
████████████████████████████████████████████████████████████████████████████████
NLP entities → Dynamic WHERE clause → Record ID whitelist
Example: COUNTRY='India' AND TROUBLE_CODE='P0500'
→ Returns [id: 12, 45, 67, 89, 112, ...]

STAGE 2: FAISS Vector Search (Cosine Similarity)
████████████████████████████████████████████████████████████████████████████████
Query → SentenceTransformer → 384-dim embedding
FAISS range_search() with IDSelector (whitelist)
Dynamic threshold: 0.30 (if SQL match) / 0.45 (no SQL match)
Returns: [(record_id, score), ...]

STAGE 3: Result Fusion + LLM Generation
████████████████████████████████████████████████████████████████████████████████
Merge SQL keyword matches + FAISS vector matches
SQL-only matches get baseline score = 0.35
Sort by score DESC → Top 5 → Fetch summaries
System prompt includes total_count for accurate counts
LLM generates streamed narrative response
```

9.2 Vector Index Architecture

Parameter	Value	Rationale
Index Type	IndexFlatIP + IndexIDMap	Exact inner product search with custom record IDs
Embedding Model	all-MiniLM-L6-v2	384-dim, fast inference, strong semantic similarity
Similarity Metric	Cosine (via normalized IP)	L2 normalized vectors → Inner Product = Cosine
Document Fields	6 concatenated text fields	subject, complaint, checked, results, repair, part
Metadata Stored	id, country, model, trouble_code, segment	Enables pre-filter ID selection
Search Method	range_search() with IDSelectorArray	Threshold-based filtering on whitelisted subset
Persistence	faiss_index.bin + vector_metadata.json	Binary index + JSON metadata on disk

9.3 Embedding Cache Strategy

To avoid redundant SentenceTransformer inference, all computed embeddings are cached in the **embedding_cache** SQLite table as BLOB-serialized NumPy arrays. The cache key is an MD5 hash of the input text, providing O(1) lookup. This is critical during ETL batch ingestion where thousands of documents are processed.

9.4 SQL Keyword Search (BM25-like)

Alongside vector search, the system performs keyword-based SQL LIKE queries across four text columns: **subject**, **customer_complaint**, **causal_parts_name**, and **summary**. Keywords are extracted by the NLP pipeline (nouns, proper nouns, adjectives) with stop-word filtering. Results are intersected with the metadata-whitelisted IDs and merged with FAISS scores.

10. Frontend Architecture

The frontend is built with Streamlit and features a multi-page layout with three main views, custom enterprise CSS theming, and interactive chat with streaming LLM responses.

10.1 Page Structure

Page	Route	Features
AI Chat & RAG	■ AI Chat & RAG	Chat interface, streamed LLM, citation cards, SQL viewer, Plotly charts, PDF export
Quality Dashboard	■ Quality Dashboard	4 KPI metric cards, trouble code chart, trend line, quality donut, model comparison
Monthly Reports	■ Monthly Reports	Year/month selector, PDF + DOCX generation, download buttons

10.2 UI Component Hierarchy

Figure 10.1 — Frontend Component Hierarchy

```
main()
  ■■■ inject_custom_styles() # Enterprise CSS injection
  ■■■ show_login_screen() # Passwordless auth form
  ■■■ show_sidebar() # Navigation + Chat sessions
  ■ ■■■ Navigation buttons # 3 page links
  ■ ■■■ New Chat / Session list # CRUD on chat_sessions
  ■ ■■■ Manual data ingest # File upload + inbox scan
  ■ ■■■ Logout button
  ■■■ render_chat_page() # Main AI chat interface
  ■ ■■■ Welcome hero + starters # Empty state with prompts
  ■ ■■■ Chat history renderer # Intent badges, tables, charts
  ■ ■■■ render_plotly_chart() # 6 chart type dispatcher
  ■ ■■■ render_citations() # Source FTIR citation cards
  ■ ■■■ Chat input handler # Query → Router → Stream
  ■■■ render_dashboard_page() # Analytics dashboard
  ■ ■■■ 4x Metric cards (SQL) # Total, Models, Dealers, Resolved
  ■ ■■■ 4x Plotly charts # Bar, Line, Donut, Table
  ■■■ render_reports_page() # PDF/DOCX report generator
```

10.3 Chat Visualization Types

Chart Type	Function	Use Case	Auto-Selected When
horizontal_bar	plot_horizontal_bar()	Rankings (dealers, codes, parts)	failures/count column detected
line	plot_line_trend()	Temporal trends	period/report_year column detected
donut	plot_donut_chart()	Category distribution	quality column + 2 columns only
histogram	plot_histogram()	Mileage distribution	mileage column detected
radar	plot_radar_comparison()	Multi-metric model comparison	COMPARE intent + resolution_rate
grouped_bar	plot_grouped_bar()	Multi-variable comparison	3+ numeric columns

11. Report Generation Engine

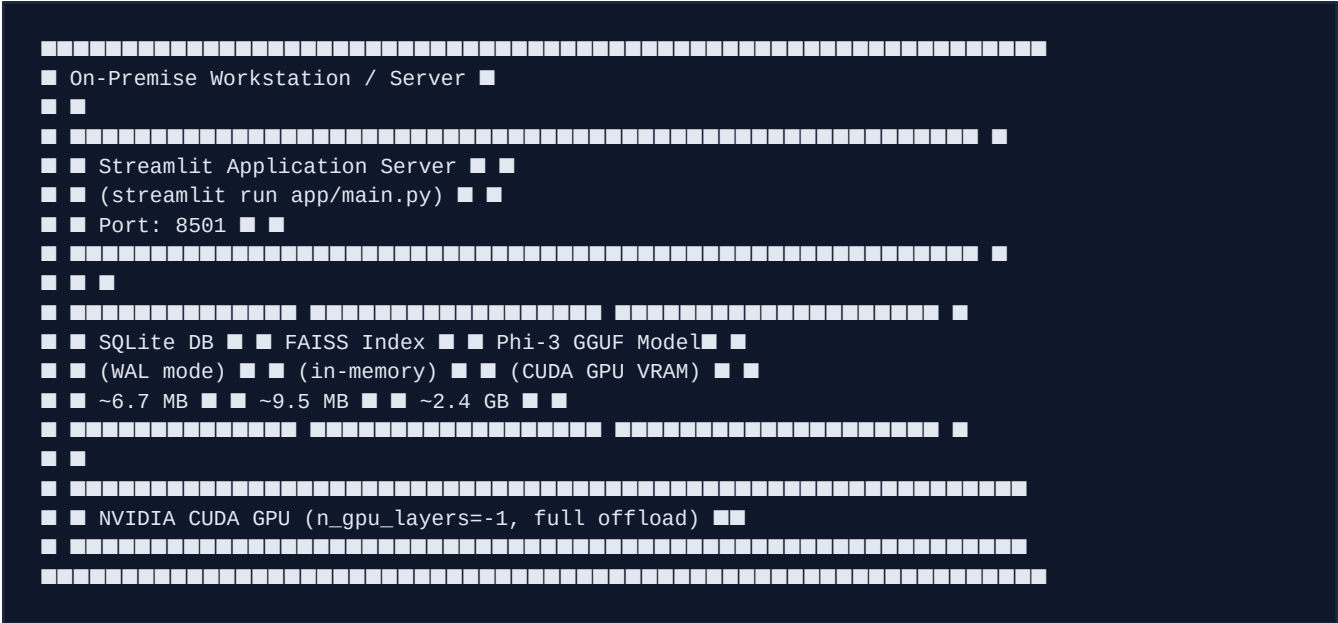
The ReportEngine in `reports/engine.py` generates two types of reports: monthly quality reports (PDF + DOCX) and chat session transcripts (PDF only). Both use enterprise-grade formatting with custom headers, footers, page numbering, and professional typography.

Feature	Monthly Report	Chat Transcript
Format	PDF (ReportLab) + DOCX (python-docx)	PDF (ReportLab) only
Header/Footer	Navy accent bar, date, page X of Y, confidential	Same enterprise canvas
Content	Executive summary, country/model tables, DTC, cases	User/assistant bubbles, SQL, tables, charts, citations
Data Source	SQLite aggregation queries (year, month)	In-memory chat_history list
Chart Embedding	Not included	Plotly → PNG → ReportLab Image
Caching	Stored in reports_cache/ directory	Generated on-demand per session

12. Deployment Architecture

The system is designed for single-node on-premise deployment on Windows or Linux workstations with NVIDIA GPU acceleration. No cloud services, API keys, or network connectivity are required.

Figure 12.1 — Deployment Topology



12.1 System Requirements

Resource	Minimum	Recommended
RAM	8 GB	16+ GB
GPU VRAM	4 GB (NVIDIA CUDA)	8+ GB
Disk Space	10 GB	20+ GB (models + data growth)
Python	3.9+	3.10+
CUDA Toolkit	11.x+	12.x
OS	Windows 10+ / Linux	Windows 11 / Ubuntu 22.04

12.2 Startup Commands

```

# Initial setup (one-time):
cd automotive_qa
pip install -r requirements.txt
python setup.py

# Launch application:
streamlit run app/main.py

```

13. Scalability Considerations

Dimension	Current Design	Scaling Strategy	Threshold
Record Count	SQLite single-file DB	Migrate to PostgreSQL with partitioning	> 500K records
Vector Index	FAISS IndexFlatIP (brute-force)	Switch to FAISS IVF-PQ or HNSW	> 1M vectors
LLM Inference	Single Phi-3 instance on 1 GPU	Model parallelism, vLLM server, or batching	> 10 concurrent users
Concurrent Users	Streamlit single-threaded	Deploy behind NGINX + multiple workers	> 5 concurrent
ETL Volume	Sequential row-by-row insertion	Batch INSERT with executemany()	> 10K rows/file
Cache Size	200-entry RAM LRU + SQLite L2	Redis with configurable TTL	> 1000 queries/day
Embedding Model	CPU + GPU inference	Pre-computed embeddings, batch encoding	> 50K documents

14. Performance Optimizations

Optimization	Implementation	Impact
Two-Tier Query Cache	L1 RAM (dict, 200 entries) + L2 SQLite (TTL 2 hours)	Eliminates duplicate LLM inference calls
Embedding Cache	SQLite BLOB storage of NumPy arrays (MD5 key)	Avoids re-encoding repeated texts during ETL
Singleton Resources	@st.cache_resource for DB, LLM, FAISS	One-time model loading across Streamlit reruns
WAL Journal Mode	PRAGMA journal_mode=WAL	Concurrent reads during writes; reduces lock contention
FAISS IDSelector	Pre-filter search space with IDSelectorArray	Reduces vector comparisons to whitelisted subset
Dynamic Threshold	0.30 (SQL match exists) vs 0.45 (no match)	Reduces false positives for unrelated queries
Adaptive Table Truncation	df.head(20) + describe() for large DataFrames	Prevents LLM context window overflow
9 B-Tree Indexes	Covering 95% of filter column patterns	Sub-millisecond SQL WHERE clause execution
GPU Full Offload	n_gpu_layers=-1 + use_mlock=True	Maximizes LLM inference throughput on NVIDIA GPU
Materialized Views	5 pre-aggregated tables refreshed post-ETL	O(1) dashboard metric queries

15. Security Considerations

Security Domain	Implementation	Assessment
Authentication	Passwordless username-based login (verify_or_create_user)	Suitable for trusted internal networks
Data Isolation	User-scoped chat sessions (user_id FK constraint)	Users can only access their own history
Data at Rest	SQLite database with OS-level file permissions	No encryption; relies on OS access controls
Data in Transit	Localhost only (no network exposure by default)	No TLS needed for localhost deployment
AI Data Privacy	Fully offline LLM — no data sent to external APIs	No data egress risk
SQL Injection	Parameterized queries (?) throughout codebase	Protected against SQL injection attacks
File Upload	Restricted to .xlsx only; saved to data/inbox/	File type validation via Streamlit uploader
LLM Output Safety	Temperature 0.1 + strict stop tokens	Minimizes hallucination; factual output
Session Management	Streamlit session_state (in-memory per browser tab)	Session cleared on logout or tab close

■ **Security Note:** The current authentication is passwordless and suitable only for trusted internal networks. For production deployment exposed to wider networks, implement password hashing (bcrypt), JWT tokens, and HTTPS via a reverse proxy (NGINX + TLS).

16. Error Handling & Resilience

Error Scenario	Handling Strategy	Module
LLM SQL generation failure	4-retry self-healing with error context injection	analytics/engine.py
LLM model file not found	FileNotFoundError with descriptive message	llm/client.py
spaCy model not installed	Fallback to regex-only NER extraction	nlp/pipeline.py
FAISS index missing	Automatic rebuild from database records	rag/embedder.py
Duplicate FTIR records	MD5 hash + UNIQUE constraint → skip silently	etl/pipeline.py
Excel column name mismatch	Column rename normalization map	etl/pipeline.py
Date parsing failure	Try 4 date formats; fallback to raw string	etl/pipeline.py
LLM summary generation failure	Heuristic 2-sentence summary fallback	etl/pipeline.py
Unrelated user query	FAISS threshold filter → general assistant fallback	core/router.py
Empty analytics result	text_stream response: 'No data available'	core/router.py
Chat session deletion	CASCADE delete of chat_history records	auth/session.py
Query cache expiry	Auto-delete from SQLite on next access	core/cache.py
Materialized view refresh failure	ROLLBACK transaction; log error	analytics/views.py
CUDA DLL not found (Windows)	Dynamic DLL directory registration at startup	core/singletons.py

17. Future Improvements

17.1 Short-Term Enhancements

- **Multi-Turn Conversation Context:** Pass previous chat messages to the LLM for context-aware follow-up answers.
- **Advanced Authentication:** Add password hashing (bcrypt), role-based access control (admin/viewer), and JWT session tokens.
- **Chart Embedding in Reports:** Render Plotly charts as PNG images and embed them in monthly PDF reports.
- **Real-Time ETL Watchdog:** Use filesystem event monitoring (watchdog library) for automatic inbox ingestion.
- **Batch ETL Optimization:** Replace row-by-row INSERT with executemany() for 10x+ ingestion performance.

17.2 Medium-Term Enhancements

- **PostgreSQL Migration:** Replace SQLite with PostgreSQL for true concurrent multi-user access and table partitioning.
- **FAISS IVF-PQ Index:** Upgrade to IVF-PQ (Inverted File with Product Quantization) for sub-linear search at 1M+ scale.
- **Larger LLM Models:** Support Qwen 2.5 7B (already downloaded) or Llama 3.1 for improved SQL generation and richer answers.
- **REST API Layer:** Expose FastAPI endpoints for headless integration with other enterprise systems.
- **Automated Testing:** Add pytest suite covering NLP classification, ETL pipeline, and analytics engine correctness.

17.3 Long-Term Vision

- **Kubernetes Deployment:** Containerize (Docker) and deploy on K8s with horizontal pod autoscaling for multi-node HA.
- **Real-Time Streaming:** Integrate with Kafka or MQTT for live field data streaming from IoT-connected vehicles.
- **Predictive Analytics:** Train time-series models (Prophet/ARIMA) on failure trends for proactive quality alerts.
- **Knowledge Graph:** Build a Neo4j knowledge graph linking models, parts, failure modes, and repair actions.
- **Multi-Language Support:** Extend NLP pipeline for Japanese, Korean, and other OEM-relevant languages.

18. Technology Stack Summary

Category	Technology	Version / Variant	Role
Language	Python	3.9+	Primary development language
Web Framework	Streamlit	1.x	Frontend UI and application server
Database	SQLite 3	WAL mode	Structured data storage
Vector Store	FAISS (faiss-cpu)	1.x	384-dim cosine similarity index
Embedding	SentenceTransformers	all-MiniLM-L6-v2	Text → 384-dim vector encoding
LLM Runtime	llama-cpp-python	CUDA build	Local GGUF model inference
LLM Model	Phi-3 Mini 4K Instruct	Q4 GGUF (2.4 GB)	Chat, summary, SQL generation
NLP	spaCy	en_core_web_sm	NER and keyword extraction
Visualization	Plotly	Express + Graph Objects	6 interactive chart types
Data Processing	Pandas	2.x	DataFrame operations and analytics
PDF Generation	ReportLab	4.x	Professional PDF report rendering
DOCX Generation	python-docx	0.8+	Word document report generation
Excel Parsing	openpyxl	3.x	FTIR Excel file reading
Numerical	NumPy	1.x	Vector array operations
GPU Acceleration	NVIDIA CUDA	11.x+	LLM inference GPU offloading

— End of Document —

Document generated on June 12, 2026 at 15:17:49